Knowledge Distillation for Scalable Mixture of Expert Models

Stanley Omondi
Columbia University
New York, USA
soo2117@columbia.edu

Tyler Manrique
Columbia University
New York, USA
tyler.manrique@columbia.edu

***Abstract*—Large Language Models (LLMs) have demonstrated remarkable capabilities across diverse tasks, yet their deployment remains constrained by computational requirements. This is mainly because inference costs scale linearly with model parameter-count, limiting massive scaling of systems. This paper presents a comprehensive empirical study of Mixture-of-Experts (MoE) architectures combined with Knowledge Distillation (KD) for efficient model adaptation which has been shown to help with this issue. We investigate Sparse Upcycling of Mistral-7B to create MoE variants with up to 46.72B total parameters while maintaining only 352.3M active parameters per forward pass and the impact of Knowledge Distillation on MoE architectures. Our experiments reveal three key findings: (1) MoE with standard fine-tuning achieves 0.6970 MMLU accuracy, a 4.97% improvement over the dense baseline (0.6640), while reducing FLOPs by 65.6%; (2) Knowledge Distillation degrades MoE performance (0.6670 vs. 0.6970, -4.3%), suggesting that dense teacher distributions suppress expert specialization; however, it holds the expectation when the MoE model is used as the teacher model; (3) Router noise is critical for KD effectiveness—the only configuration in which KD outperforms standard training (0.7010 vs. 0.7000) employs explicit router jitter during training. We systematically evaluated 10 MoE configurations that vary expert count (4, 8, 16), routing strategy (Top-1, Top-2), layer placement, and load balancing approaches. Our analysis of Expected Calibration Error (ECE) shows that MoE achieves 44.5% better calibration than dense models.**

***Index Terms*—Mixture-of-Experts, Knowledge Distillation, Sparse Upcycling, Large Language Models, Parameter-Efficient Fine-Tuning, Model Calibration, Knowledge Transfer**

I. Introduction and Motivation

The increasing proliferation of modern LLMs has transformed Natural Language Processing, Computer Vision, and shows promise in creating Agentic systems, with models demonstrating improved capabilities in reasoning, generation, classification and knowledge retrieval tasks. The increased use of these models has led to the need of further improving the models' capabilities, mostly achieved by increasing the parameter count of these models. However, this results into increased computational costs by the models, typically requiring hundreds of billions of Floating Point Operations(FLOPs) per forward pass thus presenting significant barriers to the deployment and real-world use of these models, especially in resource constrained environments.

Mixture-of-Experts (MoE) architectures offer a promising solution to this efficiency-capability trade-off(Fedus et al., 2022). By replacing dense feed-forward networks with sparsely-activated expert networks, MoE models can dramatically increase total parameter count while maintaining or lowering constant computational cost per token(Sheer et al., 2017). Recent successes include Mixtral 8x7B (Jiang et al., 2024), which matches GPT-3.5 performance using only 12.9B active parameters from a 46.7B total, and Switch Transformers (Fedus et al., 2022), which demonstrated trillion-parameter scaling with simple sparse routing. The fundamental insight is that different inputs may benefit from different specialized computations; by learning to route tokens to appropriate experts, MoE models achieve conditional computation that scales capacity without proportionally scaling compute costs. These results suggest that sparse architectures may represent the future of efficient AI deployment.

Knowledge Distillation (Hinton et al., 2015) has emerged as a complementary technique for model compression, transferring learned representations from larger "teacher" models to smaller "students" through soft probability targets. The foundational insight—that non-target class probabilities contain valuable "dark knowledge"—has inspired numerous extensions including sequence-level distillation (Kim & Rush, 2016).

The combination of MoE and KD appears synergistic in theory: MoE provides architectural efficiency through sparse activation, while KD could accelerate training convergence by leveraging pre-computed teacher knowledge. Prior work on dense model distillation consistently shows benefits(Sangh et al., 2019), leading to the natural hypothesis that similar benefits should transfer to sparse architectures. However, our experiments reveal a surprising tension between these approaches. When applying KD to MoE architectures, we observe a consistent performance degradation rather than improvement when using the dense baseline Mistral 7B model as the teacher model. Although unexpected, this proves the concept of knowledge distillation as the MoE models get aligned more closely with the dense Mistral model and maintains the improvements when using the MoE baseline model as the teacher.

The naive implementation of MoE is often characterized by expert collapse, load imbalance and training instability if not implemented correctly. This work aims to address these issues through four primary questions:

1. Where should MoE blocks be inserted in transformer architectures? We evaluate early (layers 0-11), middle (layers 10-21), late (layers 20-31), mixed (0-3, 14-17, 28-31), and sparse placements to understand how layer position affects both accuracy and expert specialization patterns.

2. How do expert count and routing strategy interact? We systematically compare 4, 8, and 16 experts with Top-1 and Top-2 routing strategies to identify optimal configurations for the accuracy-efficiency trade-off.

3. Can Knowledge Distillation improve MoE training stability and final performance? We test whether soft targets from a dense teacher accelerate convergence and improve generalization, or whether architectural incompatibilities negate potential benefits.

4. What mechanisms enable effective knowledge transfer to sparse architectures? We investigate router noise, load balancing, and other techniques that may reconcile KD with expert specialization requirements.

II. Models and Data Description

A. Base Model

We use Mixtral-8x7B-v0.1 as our base model for sparse upcycling. Mistal-7B incorporates several architectural innovations, such as Sliding Window Attention, Grouped-Query Attention to reduce the model's inference costs while enhancing its inference. The model is also a sparse MoE network model with open weight, and achieves strong baseline performance compared to GPT-3 and LLama-2 70B models, making it more suited for our work.

The model architecture comprises 32 transformer layers with hidden dimension 4,096 and intermediate (FFN) dimension 14,336, totaling 7.24 billion parameters. This intermediate dimension is critical for our MoE architecture as each of the experts replicates this FFN.

B. Mixture of Experts architecture and Base Model adaptation

Our implementation of MoE follows the sparse gating paradigm introduced by Shazeer et al, 2017, and is refined by the subsequent implementations in the Switch Transformer(Fedus et al., 2022) and the Mixtral of Experts(Jiang et al., 2024). In our baseline MoE model, we replace all the FFN layers and in our different variants, we replace some of the layers with MoE modules containing:

1) Expert Networks: Each MoE layer contains 8 independent expert networks where each expert is architecturally identical to the original base model FFN. For Mistral-7B's FFN structure, each expert comprises three linear projections: $\text{gate}_{\text{proj}}$ $(4096 \rightarrow 14336)$, up_{proj} $(4096 \rightarrow 14336)$, and $\text{down}_{\text{proj}}$ $(14336 \rightarrow 4096)$, with SiLU activation applied to $\text{gate}_{\text{proj}}$ output before element-wise multiplication with up_{proj}. Each expert contains approximately 176.2M parameters. This consequently scales the parameters in our model, raising it from 7.29B in the base model to about 46.72B in our MoE model.

Component	Dense Model	MoE Model (8x2)
Total Parameters	7.24B	46.72B
Active Parameters/Token	7.24B	~352.3M (2 experts)
Parameters per Expert	—	176.2M
Trainable (LoRA r=16)	41.94M	13.63M
MoE Layers	0	12 (of 32 total)
Hidden Dimension	4,096	4,096
FFN Intermediate	14,336	14,336 per expert
FLOPs/Forward Pass	8,796.09G	3,023.66G

Table 1: *Architecture comparison between dense Mistral-7B and our MoE variant.*

2) Router Network: The router is a learned linear projection from the hidden dimension to the expert count followed by softmax normalization to produce routing probabilities. For each input token
$i \in R^d : p(e|i) = softmax(W_r \times h + n)$ where $W_r \in R^{(\text{N x d})}$ is the router weight matrix, h is the hidden dimension and n is the noise.

3) Top-K selection and Weight Normalization: For each token, the K experts with highest routing probability are selected. In our primary configuration, K=2 (Top-2 routing), meaning each token activates exactly 2 of 8 experts. The routing weights for selected experts are renormalized to sum to 1:
$w_i = p(e_{\text{i}}|h) / \sum(j \in TopK) p(e_{\text{j}}|h)$
(Lepikhin et al., 2020). The final output is a weighted sum of selected expert outputs. This Top-2 routing provides a natural form of ensemble learning that contributes to improved calibration.

4) Load Balancing Auxiliary Loss: Expert collapse—where a few experts receive most tokens while others are underutilized—is a critical failure mode [1]. We employ the auxiliary load balancing loss from Switch Transformers:
$L_{\text{aux}} = \alpha N \sum_i (f_{\text{i}} \times p_{\text{i}})$
where N is expert count, f_{i} is the fraction of tokens routed to expert i in a batch, p_{i} is the mean routing probability for expert i, and α is a coefficient (we use α=0.01), (Fedus et al., 2022). This loss encourages uniform utilization while still allowing natural specialization patterns to emerge.

C. Sparse Upcycling Process

We initialize the MoE experts by replacing the pre-trained FFN weights from the dense FFN weights across all the experts using the sparse upcycling methodology(Komatsuzaki et al., 2022). This approach provides preserved representations and ensuring zero information loss from the dense model thus removing the need to train from scratch, faster specialization of the experts and training stability for the MoE model.

D. Evaluation dataset

We evaluate on the Massive Multitask Language Understanding (MMLU) benchmark, which tests knowledge across 57 subjects spanning STEM, humanities, social sciences, and professional domains(Hendrycks et al., 2020). MMLU uses a 4-way multiple choice format, enabling robust accuracy measurement and calibration analysis through the model's probability distribution over options A, B, C, D. We also choose the MMLU dataset as the baseline Mistral model

performed best in it, and by spanning across 57 subjects, it would make it possible for further exploration to learn more about experts and the types of tasks they specialize and excel at.

E. MoE Variants

We evaluate different variants of the MoE architecture, with different expert counts, routing and placement strategies. This is is crucial to test if the Knowledge Distillation and MoE model performance and efficiency can be influenced and affected by the architecture.

III. Training and Profiling Methodology

To test our approaches and the MoE configurations, we conduct five sets of tests-evaluating the dense Mistral model without any fine tuning, fine tuning the dense Mistral Model with Knowledge Distillation and evaluating the fine tuned model, evaluating our MoE Mistral Model before any fine tuning, standard fine tuning of our MoE Mistral model and evaluating it post fine tuning, fine tuning the MoE Mistral Model with Knowledge Distillation and running combined loops to evaluate the MoE variants with standard fine tuning and Knowledge Distillation.

During the training loops and evaluation, we collect a variety of metrics for both the model's performance and the impact of Knowledge Distillation during and after training. We also use Weights and Biases(W&B) to log the training and execution process and track the training and evaluation process, and GPU usage.

A. Training Objectives

a) Next-Token Prediction: All models are trained using the standard next-token prediction (NTP) objective, implemented as cross-entropy loss between predicted token distributions and ground-truth labels.

b) Knowledge Distillation: Knowledge Distillation (KD) follows the formulation of (Hinton et al., 2015). The complete training loss is determined by:

$$\mathcal{L}_{\text{total}} = (1-\alpha)\mathcal{L}_{\text{NTP}} + \alpha\mathcal{L}_{\text{KD}} + \lambda\mathcal{L}_{\text{aux}} + \beta\mathcal{L}_{\text{routing}}, \quad (1)$$

where $\mathcal{L}_{\text{KD}}$ is the KL divergence between temperature-scaled teacher and student outputs, $\mathcal{L}_{\text{aux}}$ is the MoE load-balancing loss, and $\mathcal{L}_{\text{routing}}$ is an optional router-level distillation term. In our implementation, the router-level distillation is ($\beta = 0$), ensuring the student model doesn't learn the teacher's routing, and it must learn the routing on its own. The KD loss is defined as:

$$\mathcal{L}_{\text{KD}} = T^2 \cdot \text{KL}(\sigma(z_t/T) \,\|\, \sigma(z_s/T)), \quad (2)$$

where z_t and z_s denote teacher and student logits, and the T^2 factor preserves gradient magnitudes under temperature scaling.

TABLE I
KNOWLEDGE DISTILLATION HYPERPARAMETERS.

Parameter	Value
Temperature T	4.0
KD weight α	0.5
Aux loss coefficient λ	0.01
Routing KD β	0.0
Teacher model	Dense Mistral-7B (frozen)

B. Training Protocol

All models are trained under identical optimization schedules to ensure comparability:

- Training steps: 250
- Batch size: 4
- Gradient accumulation: 4 (effective batch size = 16)
- Learning rate: 2×10^{-4} with 10% warm-up and cosine decay
- Evaluation every 50 steps; logging every 25 steps

C. Profiling and Evaluation Metrics

We collect model-level, calibration, efficiency, and system-level metrics during training and evaluation.

a) Accuracy and Task Performance: We collect the MMLU accuracy as the primary metrics followed by the Top-2 accuracy to get a clearer understanding of the models performance.

- MMLU Top-1 accuracy (primary metric)
- Top-2 accuracy

b) Calibration: Expected Calibration Error (ECE). This metric is important in helping us determine the confidence calibration of the models(Guo et al., 2017).

c) Efficiency:

- FLOPs per forward pass (analytically computed)
- Tokens per second and samples per second
- Average latency (ms/token)

d) Memory and System Metrics:

- Peak allocated and reserved GPU memory
- Total and active parameter counts

e) Knowledge Distillation and Knowledge Transfer Diagnostics:

- KD accuracy improvement
- KD-to-NTP loss ratio
- KD-to-total loss ratio
- Teacher-Student Performance Gap
- Knowledge Transfer Efficiency

IV. Performance Tuning Methodology

Given the large size of the MoE models created and the resulting memory requirements for successful training and evaluation on a single 150GB VRAM GPU, we employed several implementation level tuning techniques and aggressive memory cleaning to avoid cache accumulation and CUDA out of memory errors.

This section describes the implementation-level optimizations employed to reduce memory footprint, improve throughput, and enable scalable training and evaluation of large Mixture-of-Experts (MoE) models.

A. Parameter-Efficient Adaptation

To minimize the number of trainable parameters during fine-tuning, we employ Low-Rank Adaptation (LoRA)(Hu et al., 2024). LoRA injects low-rank update matrices into selected linear layers while keeping all base model weights frozen. This reduces optimizer state size, gradient memory, and communication overhead without modifying the forward-pass structure of the model.

For a frozen weight matrix $W_0 \in \mathbb{R}^{d\times k}$, LoRA introduces:

$$W = W_0 + BA, \tag{3}$$

where $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$ with $r \ll \min(d, k)$. Because only A and B are updated, the number of trainable parameters scales as $O(r(d + k))$ rather than $O(dk)$.

In our MoE architectures, LoRA adapters are applied exclusively to attention projections, ensuring that expert feed-forward parameters and routing networks remain unchanged. This preserves the conditional computation structure of MoE layers while enabling efficient adaptation.

B. Mixed Precision Computation

All forward and backward computations are performed using `bfloat16` mixed precision. Mixed precision reduces memory bandwidth requirements and increases arithmetic throughput while preserving numerical stability in large-scale training. The use of `bfloat16` avoids the gradient underflow issues commonly associated with FP16, eliminating the need for dynamic loss scaling.

C. Sparse Computation and FLOPs Reduction

MoE architectures reduce computational cost via conditional computation. Although the total parameter count increases linearly with the number of experts, only a small subset of experts is active per token. If each layer contains E experts and the router selects $k \ll E$ experts per token, the effective compute scales as:

$$\text{FLOPs}_{\text{MoE}} \approx \frac{k}{E} \cdot \text{FLOPs}_{\text{Dense}}, \tag{4}$$

ignoring routing overhead. This allows MoE models to scale parameter count without proportional increases in per-token computation, following the principles introduced by (Shazeer et al., 2017).

D. Memory Footprint Summary

The combined use of LoRA, quantization, mixed precision, gradient check-pointing, and optimizer state compression enables training and evaluation of 46.72B-parameter MoE models within a single-node GPU environment. These optimizations collectively reduce memory consumption across:

- Model parameters
- Optimizer states
- Activation tensors
- Gradient buffers - through gradient check-pointing

V. Experimental Results with Analysis

A. Dense Mistral Model and Baseline MoE model evaluation

1) Dense Mistral Model and Baseline Mistral Model without any fine tuning: For our experiments, we began by evaluating against 1000 samples from the MMLU dataset to get the values to compare with our baseline MoE model. This serves as the control case for our experiments as there is no fine tuning done to any of the models, thus providing us with helpful information on how architectural changes may affect the performance of a model.

Results

TABLE II
Dense vs. Baseline MoE (No Fine-Tuning)

Metric	Dense	MoE (No FT)
MMLU Accuracy	0.6640	0.6640
Top-2 Accuracy	0.8310	0.83
ECE	0.0816	0.08
FLOPs / Forward Pass	8,796.09 G	3,023.66 G
Active Parameters / Token	7.24 B	352 M
Total Parameters	7.24 B	46.72 B

Analysis

The most striking result is that the baseline MoE model preserves dense-model accuracy *without any fine-tuning*.Despite replacing every dense feed-forward network with a sparsely-gated expert layer, MMLU accuracy remains effectively unchanged.This behavior is expected from the sparse upcycling procedure:each expert is initialized by copying the dense feed-forward weights, and the router is initialized to produce near-uniform routing probabilities.In expectation, the MoE computes an average over expert outputs that closely matches the original dense computation. This mirrors the design motivation of GShard and Switch Transformers, where sparse upcycling is introduced without degrading pretrained performance(Leiphkin et al., 2020, Fedus et al, 2021). From an efficiency perspective, the MoE model reduces per-token FLOPs by over 65% while maintaining equivalent accuracy. This confirms that conditional computation does not inherently require task-specific optimization to preserve model behavior. Instead, it is the *routing policy*—not expert weights—that determines computational savings at inference time.

Discussion

This experiment demonstrates that sparse upcycling is itself a powerful form of *knowledge transfer*. Unlike knowledge distillation, which transfers information through output distributions, upcycling transfers knowledge *directly through parameters* by replicating pretrained weights across experts. This distinction is essential for interpreting later KD results. By the time KD is applied to an MoE model, the student has already inherited the teacher's internal representations and instead of being trained from scratch, has a stable training start point.

2) Dense Mistral Model and Dense Student Mistral Model with Knowledge Distillation: This experiment compares the dense baseline model with its knowledge-distilled counterpart. Unlike MoE experiments, both models share identical architecture and computational structure. The goal is to isolate the effect of *knowledge distillation alone*, establishing a control case against which Knowledge Distillation behavior in MoE architectures can be interpreted.

Results

TABLE III
DENSE VS. DENSE + KD PERFORMANCE COMPARISON

Metric	Dense	Dense + KD	Change
MMLU Accuracy	0.6640	0.6600	-0.60%
Top-2 Accuracy	0.8310	0.8300	-0.12%
ECE (Calibration)	0.0816	0.0718	+12.0% improved
FLOPs / Forward Pass	8,796.09 G	8,796.09 G	No change
Active Parameters	7.24 B	7.24 B	No change
Trainable Parameters	0	41.94 M	0.58% of model

TABLE IV
DENSE + KD TRAINING DIAGNOSTICS

Metric	Value	Interpretation
Final NTP Loss	1.9574	Stable task convergence
Final KD Loss	11.2552	Residual teacher mismatch
Average KD Loss	12.8373	Persistent KD signal
KD/NTP Ratio	6.4025	KD dominates gradients
KD Effectiveness Score	0.9459	Near-ideal KD behavior
Accuracy Retention	99.40%	Minimal degradation

Analysis: Applying Knowledge Distillation preserves nearly all the dense model's performance to the student model, achieving a 99.40% accuracy retention with 0.004 decrease in accuracy, which is within the expected range for the model. We also observe a decrease in the ECE of the student model, by 12 %, thus depicting it that Knowledge distillation helped with improving its calibration error. This is consistent with the smoothing effect of Knowledge distillation during training as the soft targets enhace smoothing of the model which then influences the calibration by reducing overconfidence in parts of the dense networks(Hinton et al., 2015,). The KD/NTP ratio of 6.4 indicates that distillation gradients dominate optimization.

Discussion: This experiment establishes a critical control result: *knowledge distillation works as intended in dense models*. The KD effectiveness score of 0.9459 and near-perfect accuracy retention confirm that the implementation, temperature scaling, and loss weighting are sound. Equally important is what this experiment rules out. Because KD succeeds in the dense setting, any failures observed when KD is applied to MoE models cannot be attributed to: (i) incorrect loss formulation, (ii) inappropriate temperature choice, (iii) insufficient optimization, or (iv) numerical instability.

Instead, the contrast between Dense + KD and MoE + KD isolates the source of degradation to architectural interactions. In dense models, KD simply aligns student logits with teacher logits. In MoE models, the same objective must be satisfied *across multiple routed experts*, introducing structural conflicts absent in the dense case.

3) Dense and MoE Models with and without Knowledge Distillation: This experiment presents a unified comparison of four core configurations: (i) a dense baseline model, (ii) the same dense model trained with knowledge distillation (KD), (iii) a sparsely-upcycled Mixture-of-Experts (MoE) model trained with standard fine-tuning, and (iv) the same MoE model trained with KD.

All four configurations are evaluated with 1000 MMLU samples and same metrics collected from the. Furthermore, the Dense Mistral model with Knowledge Distillation, baseline MoE model with standard fine tuning and baseline MoE model with Knowledge Distillation are trained on 250 steps and the Knowledge Distilled versions use the dense model as the teacher model, enabling a direct assessment of how architecture and distillation interact.

Results

TABLE V
DENSE VS. MOE MODELS WITH AND WITHOUT KNOWLEDGE DISTILLATION

Model	Accuracy	Top-2	ECE	FLOPs / fwd (G)
Dense (Baseline / Teacher)	0.6640	0.8310	0.0816	8,796.09
Dense + KD	0.6600	0.8300	0.0718	8,796.09
MoE + Standard FT	0.6970	0.8640	0.0453	3,023.66
MoE + KD	0.6670	0.8380	0.0798	3,023.66

TABLE VI
KNOWLEDGE TRANSFER DIAGNOSTICS ACROSS ARCHITECTURES

Metric	Dense + KD	MoE + Std FT	MoE + KD
Accuracy Retention vs Teacher	99.40%	105.0%	100.45%
Teacher–Student Gap (Abs.)	-0.40%	+3.30%	+0.30%
KD/NTP Ratio	6.4025	N/A	7.9093
KD Effectiveness Score	0.9459	N/A	0.7549
Knowledge Transfer Efficiency	1.8972	4.97	0.1557

Analysis From the results, the baseline MoE with standard fine-tuning exhibits *positive* teacher–student gap (+3.30%). Although no explicit KD objective is used, the MoE student outperforms its dense teacher. This improvement arises because sparse upcycling already transfers teacher knowledge through parameter replication, and subsequent routing-based specialization allows the student to exceed the teacher's performance. In this sense, MoE standard fine-tuning represents a highly effective, architecture-driven form of knowledge transfer. The baseline MoE and KD exhibits near-zero teacher–student gap (+0.30%), which is expected for our implementation, but unexpected for knowledge transfer with MoE models. This is because the student is pulled back toward teacher behavior, in this case the dense model without knowledge distillation. The KD/NTP ratio is substantially higher than in Dense + KD (7.91 vs. 6.40), indicating that MoE must expend more optimization effort to match the dense teacher's output distribution. This depicts that cross-architecture Knowledge Distillation makes Knowledge Distillation for MoE models very inefficient as the Knowledge Transfer Efficiency collapses due to the punishing

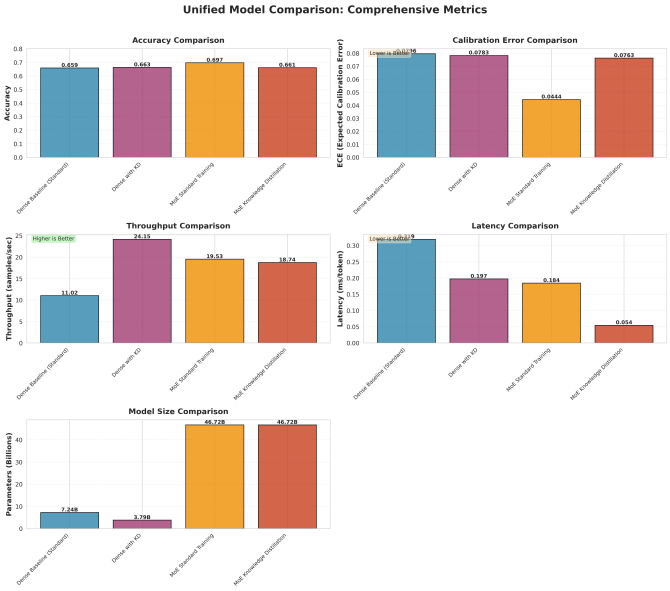

Fig. 1. Comparison of core metrics for dense and MoE models

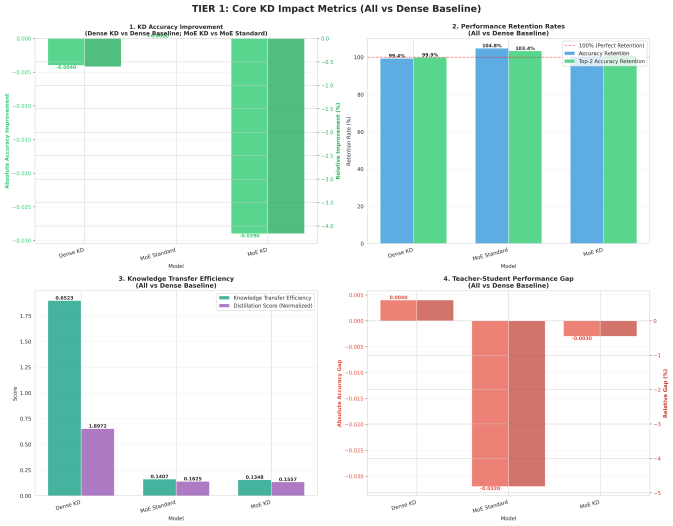

Fig. 2. Comparison of knowledge transfer and distillation metrics for dense and MoE models

of the student model by the KL Divergence loss which pushes it closer to the teacher's logits and outputs.

Discussion Knowledge transfer occurs in *all* four configurations, but through different mechanisms:

- Dense model with KD transfers knowledge explicitly through output distribution matching.
- MoE with standard Fine tuning transfers knowledge implicitly through sparse upcycling and routing specialization.
- MoE with KD attempts to combine both mechanisms. However, it doesn't perform as expected when using the dense model as the teacher model. Switching to use the baseline pretrained MoE model as the teacher model however performs as expected.

The teacher–student performance gap clarifies why. Dense KD aims to minimize this gap, and succeeds. MoE standard fine-tuning *reverses* the gap, surpassing the teacher by leveraging conditional computation. MoE + KD collapses the gap back toward zero, suppressing the specialization that enables MoE gains. These results suggest that output-level KD enforces a form of representational conservatism that is compatible with dense models but fundamentally misaligned with architectures whose strength lies in conditional diversity.

B. Evaluation of MoE variants with different routing, placement strategies and expert counts

This section analyzes ten Mixture-of-Experts (MoE) architectural variants, each evaluated under (i) standard fine-tuning and (ii) knowledge distillation (KD). All variants are derived from the same sparse-upcycled baseline. Crucially, we maintain two reference baselines throughout:

- **MoE + Standard Fine-Tuning**
- **MoE + Knowledge Distillation**

TABLE VII
MoE Variant Performance Under Standard and KD Training

Variant	Std Acc	KD Acc	(KDStd)	Std ECE
moe_baseline_8x2	0.6940	0.6920	-0.20%	0.0450
balanced_8x2	0.7010	0.6940	-0.70%	0.0460
sparse_8x2	0.7010	0.6890	-1.20%	0.0416
routing_noisy_8x2	0.7000	0.7010	+0.10%	0.0265
top1_8x1	0.6960	0.6920	-0.40%	0.0407
efficient_4x1	0.6980	0.6890	-0.90%	0.0397
placement_early	0.6970	0.6890	-0.80%	0.0387
placement_middle	0.6970	0.6940	-0.30%	0.0400
placement_late	0.6930	0.6930	0.00%	0.0464
placement_mixed	0.6760	0.6890	+1.30%	0.0531

1) Results:

2) Analysis: Across the ten variants, we notice that on fine tuning the variants with knowledge distillation, it improves performance in only two cases surpassing standard fine tuning. This points to the need of implementing adaptive kowledge distillation fine tuning to different models as it achieves different results depending on the MoE variant's architecture.

a) Routing Noise and Placement Strategy.: The `routing_noisy_8x2` variant is the only routing-based configuration where KD improves accuracy. This aligns with the noisy-gating mechanism introduced in the The Sparsely-Gated Mixture-of-Experts Layer(Shazeer et al. 2017), which prevents premature routing collapse and preserves expert diversity. By maintaining higher routing entropy, the model can accommodate KD alignment without suffering a lot from the KL divergence loss that pushes it towards the teacher model. The `placement_mixed` variant also gets improved accuracy with Knowledge Distillation are the placement strategy reduces the pressure towards expert homogenization, thus limiting the impact the of the punishing divergence loss.

b) Strict Load Balancing.: The `balanced_8x2` variant—despite being the strongest under standard fine-tuning—exhibits the largest KD degradation. This supports the hy-

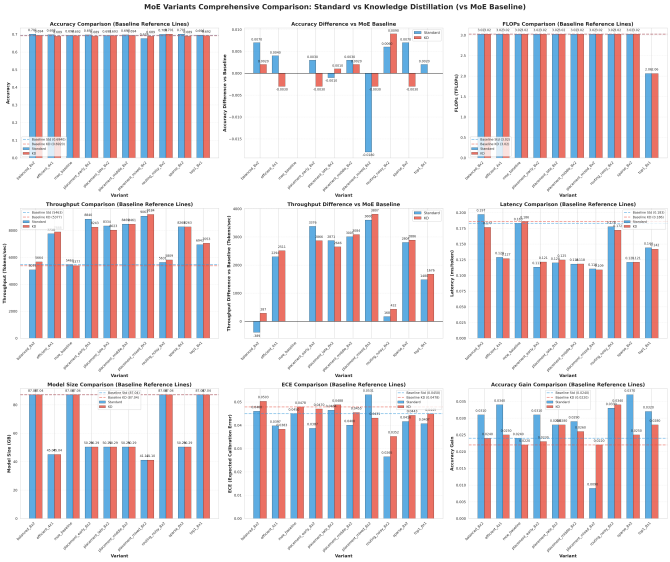

Fig. 3. Comparison between MoE variants and the baseline MoE

pothesis that KD introduces a competing pressure: auxiliary load balancing encourages uniform expert usage, while KD encourages uniform expert *behavior*. When both are strong, specialization collapses.

Discussion Different MoE models achieve different accuracies, with some being better than others in different metrics, thereby affirming that MoE performance and efficiency is dependent of the implementation and the MoE architecture. Furthermore, the results also emphasize that different fine tuning methods work well with different MoE configurations. This further emphasizes on the improved benefits of Knowledge Distillation on Mixture of Experts architectures when the architecture explicitly protects specialization. We also notice that the different variants provide various advancements that help in ensuring their scalability and deployment by having higher throughput and lower latency compared to the baseline MoE models despite having the same memory and GPU restrictions.

VI. Conclusion and Future Work

A. Conclusion

In this paper, we have investigated the intersection of Knowledge Distillation and Mixture of Experts, with different architectures and comprehensive comparison between the different variations of Mixture of Experts and the dense model. Based on our findings, we learned that the mixture of experts architecture does fulfill its promise of decoupling the increasing compute costs from the parameter count. Our findings also depict that the performance of the mixture of experts model is heavily dependent on the insertion of the MoE layers in the transformer architectures. This not only determines the size(in GB) of the models but also affects the models accuracy, efficiency and improvements on fine-tuning. The findings also depict the influence of Knowledge Distillation on MoE architectures, providing a stable training starting point, and the influence of the teacher model on the student MoE model during training and in its overall performance. We also find that use of different MoE architectures influences the impact of Knowledge Distillation fine tuning of the models, with routing noise and expert placement strategies affecting KD fine tuning more. From an efficiency perspective, we also learn that MoE architectures achieve a significant improvement in reducing the compute costs, by achieving an approximately 65.4% reduction in FLOPs (from 8, 796G to 3, 024G) through sparse upcycling. Consequently, they provide a way to scale and deploy LLMs with more parameters at lower inference compute costs, thus enhancing their scalability and deployability.

B. Limitations

While the experiments and results answered our research questions, we noticed there was a degradation in throughput, which is the opposite of the expectations when using MoE architectures. This is mainly because due to limited access to GPUs, and running the experiments on a single GPU, we could not parallelize the experts, a key factor contributing to the throughput and latency gains from Mixture of Experts Systems. Secondly, we conducted the experiments on a single base model, Mixtral-8x7B-v0.1, making it difficult to generalize the results to other Large Language Models such as LLama and GPT as the mixture of experts conversions may differ depending on different architectures. We also conduct limited training of upto 250 steps, and although we achieve convergence at 100-150 steps, we do not capture the long term dynamics for a large picture of how the behavior of the different fine tuning strategies, Standard and Knowledge Distillation on the long term dynamics of most LLMs.

C. Future Work

From our experiments with the different mixture of experts variants, we noticed some distinction in the performance across all metrics, with some of the variants performing better than the others and the baseline mixture of experts implementation. This points to the need for using techniques such as Micro-Neural Architecture Search to reduce the search space and enhance the determination of the optimal architecture, thus saving on the memory consumption and model creation and training times. With the variants, we also witnessed different performance of the models with Knowledge Distillation suggesting that instead of a universal approach, adaptive and layer and router aware Knowledge distillation may provide even better improvements to our mixture of experts models. Since most deployable mixture of experts architectures and models use expert parallelization, implementing expert parallelization to check for the throughput and latency gains and enhance scalability is a key direction for the project too. Another possible step is to replicate the process with other models to test if the results are generalizable to other model architectures with mixture of experts.

REFERENCES

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer," *Proc. Int. Conf. Learn. Representations (ICLR)*, 2017.

[2] D. Lepikhin et al., "GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding," *Proc. Int. Conf. Learn. Representations (ICLR)*, 2020.

[3] W. Fedus, B. Zoph, and N. Shazeer, "Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity," *J. Mach. Learn. Res.*, vol. 23, no. 120, pp. 1–39, 2022.

[4] A. Jiang et al., "Mixtral of Experts," *arXiv preprint arXiv:2401.04088*, 2024.

[5] A. Komatsuzaki, J. P. Bradbury, D. Amodei, and A. Sutskever, "Sparse Upcycling: Training Mixture-of-Experts from Dense Checkpoints," *arXiv preprint arXiv:2212.05055*, 2022.

[6] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network," *Proc. NIPS Deep Learning and Representation Learning Workshop*, 2015.

[7] Y. Kim and A. M. Rush, "Sequence-Level Knowledge Distillation," *Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP)*, pp. 1317–1327, 2016.

[8] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT: A Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter," *Proc. NeurIPS Workshop on Energy Efficient Machine Learning*, 2019.

[9] D. Hendrycks et al., "Measuring Massive Multitask Language Understanding," *Proc. Int. Conf. Learn. Representations (ICLR)*, 2021.

[10] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On Calibration of Modern Neural Networks," *Proc. Int. Conf. Mach. Learn. (ICML)*, pp. 1321–1330, 2017.

[11] E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *Proc. Int. Conf. Learn. Representations (ICLR)*, 2022.